

June 25, 2026

```
[2]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
```

```

from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[3]: currentversuch = "241"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\241

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\241\Diagramme

Delete UTF8 False encoding

```

[4]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a] '
#     text=re.sub(patterna,"ä",text)
#     patternu=r'[][u] '
#     text=re.sub(patternu,"ü",text)
#     patterno=r'[][o] '
#     text=re.sub(patterno,"ö",text)
#     return text

def deletevocals(text):

```

```

# Einrückungen entfernen
text = textwrap.dedent(text)
# entfernt Bindestriche
text = re.sub(r'\n\s*', "", text)
# entfernt Zeilenumbrüche
text = text.replace("\n", " ")

# Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
def convert(match):
    # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
    ↳ 'A')
    buchstabe = match.group(1)
    # Dictionary für die echten Umlaute
    umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
    # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
    ↳ wie es ist
    return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
    ↳ Auswahl
    return re.sub(r'``([auoAUO])', convert, text)

```

```

[421]: print(deletevowels("""
"""))

```

Runden signifikanter Stellen

```

[6]: def round_to_sigs(val, errVal, einheiten):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
    ↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
    ↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
    ↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter
    -----
    **val** : float
        Messwert

    **errVal** : float
        Ungenauigkeit des Messwertes
    """

```

```

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
    """

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
    exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *
↳scale # mit scale wird das Komma so verschoben, dass alle
↳signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳* scale

qty = f"\qty{{{val_round}}({err_round})}{{{einheiten}}}"
num = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10^/e Prefixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```
[ ]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamaters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
    ↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

    absolute_err=sp.simplify(sp.sqrt(error),rational = True)
    relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)
```

```

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[8]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
    ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
        exp -= 1
        scale = Decimal(10) ** exp
        abweichung_round = (abweichung / scale).quantize(Decimal('1'),
        ↪rounding=ROUND_CEILING) * scale
        res = f"\\num{{{abweichung_round}}}"

    return float(abweichung_round),res

```

```

[9]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2

```

```

#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
    ↪val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
            Messreihe & {nam1}[\\unit{{{einheiten}}}] &
    ↪{nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
    ↪\\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\\\midrule
        """).strip()

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
    ↪err2_array):

        # Deine Berechnungen (bleiben exakt gleich)
        VAL1 = round_to_sigs(val1, err1, r'')[2]
        VAL2 = round_to_sigs(val2, err2, r'')[2]

        abs_val = np.abs(val1 - val2)
        abs_delta = np.sqrt(err1**2 + err2**2)

        if abs_delta != 0:
            SIGMA = sigma_abweichung(val1, err1, val2, err2)[1]
        else:
            SIGMA = 0.0

        ABS = round_to_sigs(abs_val, abs_delta, r'')[2]

        rel = abs_val / np.abs(val1) * 100 if val1 != 0 else 0
        rel_delta = rel * np.sqrt((abs_delta / abs_val)**2 + (err1 / val1)**2)
    ↪if abs_val != 0 and val1 != 0 else 0
        REL = round_to_sigs(rel, rel_delta, r'')[2]

        # Eine saubere Zeile nur mit den Werten für LaTeX
        row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\\"
        table_rows.append(row)

```

```

body_str = "\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[115]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r'')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else
↪0
#     REL=round_to_sigs(rel,rel_delta,r'')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
#         \\toprule
#         ↪
↪{nam1}[\\unit{{{{einheiten}}}}]&{nam2}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
↪}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
↪}}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\midrule
#         {VAL1}&\\num{{{{val2}}}}&{ABS}&{REL}&{SIGMA}\\\\\\
#         \\bottomrule
#         \\end{{tabularx}}
#         \\label{{table:Vergleich_{nam1}}}
#         \\end{{table}}
#     """)
#     print(output)

```



```

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    VAL2=round_to_sigs(val2,err2,r'[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

```

I Zeitkonstanten

```
[150]: T_12=np.array([316.0,40.4,32.0])
Delta_T_12=np.array([4,2,0.8])*np.sqrt(2)
T_12,Delta_T_12,latexT12,_=v_round(T_12,Delta_T_12,r'\micro\s')
R=np.array([1,10,1])
Delta_R=0.05*R
_,_,latexR,_=v_round(R,Delta_R,r'\kilo\ohm')
C=np.array([470.0,4.7,47])
Delta_C=0.1*C
_,_,latexC,_=v_round(C,Delta_C,r'\nano\farad')

tau_theo=R*C
Delta_tau_theo=tau_theo*np.sqrt(Delta_R**2/R**2 + Delta_C**2/C**2)
tau_theo,Delta_tau_theo,latextau_theo,_=v_round(tau_theo,Delta_tau_theo,r'\micro\s')
tau_exp=T_12/np.log(2)
Delta_tau_exp=Delta_T_12/np.log(2)
tau_exp,Delta_tau_exp,latextau_exp,_=v_round(tau_exp,Delta_tau_exp,r'\micro\s')

for i in range(3):
    □
    ↪S=sigma_abweichung(tau_theo[i],Delta_tau_theo[i],tau_exp[i],Delta_tau_exp[i])[0]
    □
    ↪print(f"{latexR[i]}&{latexC[i]}&{latexT12[i]}&{latextau_theo[i]}&{latextau_exp[i]}&{S}\\\\"
\\num{1.00(0.05)}&\\num{470(50)}&\\num{316(6)}&\\num{470(60)}&\\num{456(9)}&0.24\\
\\num{10.0(0.5)}&\\num{4.7(0.5)}&\\num{40(3)}&\\num{47(6)}&\\num{58(5)}&1.5\\
\\num{1.00(0.05)}&\\num{47(5)}&\\num{32.0(1.2)}&\\num{47(6)}&\\num{46.2(1.8)}&0.13\\
```

```
[12]: gff("R*C", "R,C")
```

Funktion:

CR

$C R$

Absoluter Fehler:

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

```
[12]: (C*R,
      sqrt(C**2*Delta_R**2 + Delta_C**2*R**2),
      sqrt(Delta_R**2/R**2 + Delta_C**2/C**2),
```

```
[(R, Delta_R), (C, Delta_C)]
```

```
[13]: gff("T_12/ln(2)", "T_12")
```

Funktion:

$$\frac{T_{12}}{\log(2)}$$

$\frac{T_{12}}{\log(2)}$

Absoluter Fehler:

$$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$$

$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$

Relativer Fehler:

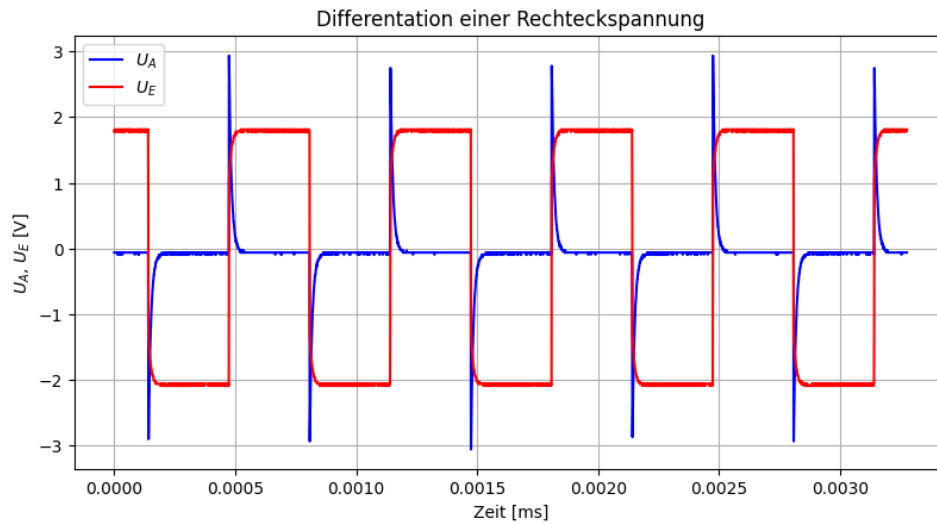
$$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$$

$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$

```
[13]: (T_12/log(2),  
      sqrt(Delta_T_12**2)/log(2),  
      sqrt(Delta_T_12**2/T_12**2),  
      [(T_12, Delta_T_12)])
```

II Differnetation einer Rechteckspannung

```
[422]: plt.close('all')  
plt.figure(figsize=(figsize_cm(12,6)))  
plt.title('Differentiation einer Rechteckspannung')  
plt.xlabel(r'Zeit [ms]')  
plt.ylabel(r'$U_A$, $U_E$ [V]')  
  
T,U_A,U_E = np.loadtxt(Messwerte_path/"A2_Viereck_DIF.txt",skiprows=10,␣  
    ↪unpack=True)  
  
T=T/(1.25*1e6)  
U_A=(U_A-126)/32  
U_E=(U_E-132)/32  
  
plt.plot(T,U_A,color='blue',label=rf'$U_A$')  
plt.plot(T,U_E,color='red',label=rf'$U_E$')  
plt.legend()  
plt.grid()  
# plt.tight_layout()  
plt.savefig(Ausgabe_path/"A2_Rechteck_Diff.png",format="png")  
plt.show()
```



III Grenzfrequenzen

```
[15]: f_exp=np.array([3.34,3.16])
Delta_f=0.2
f_exp,Delta_f,latexf,_=v_round(f_exp,Delta_f,r'\hertz')
C=47
Delta_C=np.array([4.7,4.7])
C,Delta_C,_,_=v_round(C,Delta_C,r'\nano\farad')
R=np.array([1.0,1.0])
Delta_R=0.1
R,Delta_R,_,_=v_round(R,Delta_R,r'\kilo\ohm')

f_g=1/(2*np.pi*R*C*1e-3)
Delta_f_g=f_g*np.sqrt(Delta_R**2/R**2 + Delta_C**2/C**2)
compare_table_array(r'f^\text{exp}_g',r'f^\text{theo}_g',r'\hertz',f_exp,Delta_f,f_g,Delta_f_g)
```

```
\begin{table}[htb]
\centering
\caption{Vergleich von  $f^\text{exp}_g$  und  $f^\text{theo}_g$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe &  $f^\text{exp}_g$ [\unit{\hertz}] &
 $f^\text{theo}_g$ [\unit{\hertz}] & \text{abs.Abw.}[\unit{\hertz}] &
\text{proz.Abw.}[\unit{\percent}] &  $S[\sigma]$  \\
\midrule
& \num{3.34(0.20)} & \num{3.4(0.5)} & \num{0.0(0.6)} & \num{1(16)} &
\num{0.09} \\
& \num{3.16(0.20)} & \num{3.4(0.5)} & \num{0.2(0.6)} & \num{7(17)} &
\num{0.5} \\
\bottomrule\end{tabular}
```

```
\end{tabularx}
\label{table:Vergleich_f^\text{exp}_g}
\end{table}
```

Plotten der Grenzfrequenzänge

```
[460]: #Laden der Daten für die Frequenzgänge
plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)),sharey=True)

freq_hoch, Vrms_hoch, Vdb_hoch, phase_hoch=np.loadtxt(Messwerte_path/
    ↪ 'A3_Hochpass_Phaseplot.txt',skiprows=1, unpack=True)
freq_tief, Vrms_tief, Vdb_tief, phase_tief=np.loadtxt(Messwerte_path/
    ↪ 'A3_Tiefpass_Phaseplot.txt',skiprows=1, unpack=True)
#Anpassen von Geraden
def double_log_linear(x, a, b):
    return a*x**b
#Grenzen für geraden Bereich
x0_tief=30
x1_tief=93
x0_hoch=30
x1_hoch=100
#Fit usw für Hochpass
#plot
ax1.set_title('Frequenzgang RC-Hochpass')
ax1.set_xscale('log')
ax1.set_yscale('log')
ax1.set_xlabel('Frequenz [Hz]')
ax1.set_ylabel('Effektivspannung $V_{rms}$ [V]')
ax1.plot(freq_hoch, Vrms_hoch, label='Hochpass Spannung')
#fit
pophoch_l,pcovhoch_l = curve_fit(double_log_linear, freq_hoch[:x0_hoch],
    ↪ Vrms_hoch[:x0_hoch])
pophoch_r,pcovhoch_r = curve_fit(double_log_linear, freq_hoch[x1_hoch:],
    ↪ Vrms_hoch[x1_hoch:])
#plotfit
ax1.plot(freq_hoch, double_log_linear(freq_hoch, *pophoch_l), '--',
    ↪ label='links',color='lightgrey')
ax1.plot(freq_hoch, double_log_linear(freq_hoch, *pophoch_r), '--',
    ↪ label='rechts',color='lightgrey')
#plt.plot(freq_tief, double_log_linear(freq_tief, *popttief), 'g--',
    ↪ label='Tiefpass
#Plot Grenzfrequenz
a_l=pophoch_l[0]
b_l=pophoch_l[1]
a_r=pophoch_r[0]
b_r=pophoch_r[1]
```

```

f_cutoff_hoch = (a_l/a_r)**(1/(b_r-b_l))
ax1.axvline(x=f_cutoff_hoch, color='red', linestyle='-', label='Grenzfrequenz_
↳Hochpass',linewidth=1)
ax1.text(f_cutoff_hoch,8,f'{f_cutoff_hoch:.2f} Hz',color='red')
# Grenzfrequenz mit 3dB Methode
index_3dB = np.abs((Vdb_hoch[130])-Vdb_hoch-3).argmin()
ax1.scatter(freq_hoch[index_3dB],Vrms_hoch[index_3dB],color='red',marker='x')
ax1.axvline(x=freq_hoch[index_3dB], color='red', linestyle='--',
↳label='Grenzfrequenz 3dB',linewidth=1)
ax1.text(freq_hoch[index_3dB]+1300,Vrms_hoch[index_3dB]-0.
↳05,f'{freq_hoch[index_3dB]:.2f} Hz',color='red')
ax1.scatter(1e2,Vrms_hoch[0],label=f'D = {Vdb_hoch[0]:.2f} dBV',color='red')

#Fehlergeraden:
# plt.axvline(x=f_cutoff_hoch-f_cutoff_hoch, color='red', linestyle='--')
# plt.axvline(x=f_cutoff_hoch+f_cutoff_hoch.s, color='red', linestyle='--')
# plt.savefig('figures/A3/Frequenzgaenge_RC_hoch.png', dpi=300,
↳bbox_inches='tight')
ax1.legend()
ax1.grid()

#Dasselbe für den Tiefpass
#plot
ax2.set_title('Frequenzgang RC-Tiefpass')
ax2.set_xlabel('Frequenz [Hz]')
ax2.set_xscale('log')
ax2.set_yscale('log')
ax2.plot(freq_tief, Vrms_tief, label='Tiefpass Spannung')

#fit
popttief_l,pcovtief_l = curve_fit(double_log_linear, freq_tief[:x0_tief],
↳Vrms_tief[:x0_tief])
popttief_r,pcovtief_r = curve_fit(double_log_linear, freq_tief[x1_tief:],
↳Vrms_tief[x1_tief:])
#plotfit
ax2.plot(freq_tief, double_log_linear(freq_tief, *popttief_l), '--',
↳label='links',color='lightgrey')
ax2.plot(freq_tief, double_log_linear(freq_tief, *popttief_r), '--',
↳label='links',color='lightgrey')
#plt.plot(freq_tief, double_log_linear(freq_tief, *popttief), 'g--',
↳label='Tiefpass')
#Plot Grenzfrequenz
a_l=popttief_l[0]
b_l=popttief_l[1]
a_r=popttief_r[0]

```

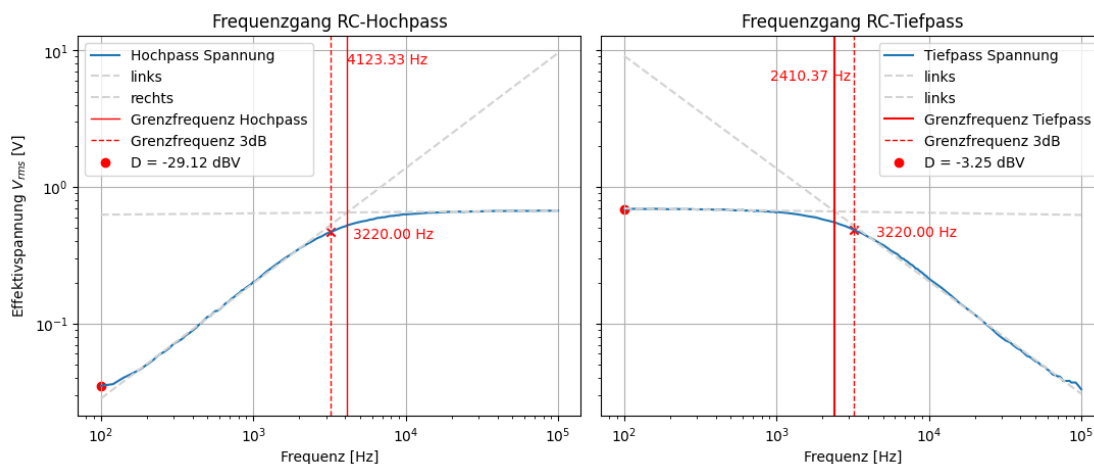
```

b_r=popttief_r[1]
f_cutoff_tief = (a_l/a_r)**(1/(b_r-b_l))
ax2.axvline(x=f_cutoff_tief, color='red', linestyle='-', label='Grenzfrequenz_
↳Tiefpass')
ax2.text(f_cutoff_tief-1500,6,f'{f_cutoff_tief:.2f} Hz',color='red')#
# Grenzfrequenz mit 3dB Methode
index_3dB_tief = np.abs((Vdb_tief[5])-Vdb_tief-3).argmin()
ax2.
↳scatter(freq_tief[index_3dB_tief],Vrms_tief[index_3dB_tief],color='red',marker='x')
ax2.axvline(x=freq_tief[index_3dB_tief], color='red', linestyle='--',↳
↳label='Grenzfrequenz 3dB',linewidth=1)
ax2.text(freq_tief[index_3dB_tief]+1300,Vrms_tief[index_3dB_tief]-0.
↳05,f'{freq_tief[index_3dB_tief]:.2f} Hz',color='red')
ax2.scatter(1e2,Vrms_tief[0],label=f'D = {Vdb_tief[0]:.2f} dBV',color='red')

#Fehlergeraden
# plt.axvline(x=f_cutoff_tief.n-f_cutoff_tief.s, color='red', linestyle='--')
# plt.axvline(x=f_cutoff_tief.n+f_cutoff_tief.s, color='red', linestyle='--')
ax2.legend()
ax2.grid()

plt.savefig(Ausgabe_path/'uselessfreqplots.
↳png',format="png",bbox_inches='tight')
plt.tight_layout()
plt.show()
# print("Hoch",unc(f_cutoff_hoch))
# print("Tief",unc(f_cutoff_tief))

```



Vergleich mit Aufgabe IX

```
[420]: compare_table(r'D_{\Rnum{3}}',r'D_{\Rnum{9}}',r'\decibel',-29.12,1,-28.85,1)
compare_table(r'D_{\Rnum{3}}',r'D_{\Rnum{9}}',r'\decibel',-3.25,1,0.19,1)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
D_{\Rnum{3}}[\unit{\decibel}]&D_{\Rnum{9}}[\unit{\decibel}]&\text{abs. Abw.}[\unit{\decibel}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{\num{-29.1(1.0)}&\text{\num{-28.9(1.0)}&\text{\num{0.3(1.5)}&\text{\num{1(5)}&\text{\num{0.20}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_D_{\Rnum{3}}}
\end{table}
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
D_{\Rnum{3}}[\unit{\decibel}]&D_{\Rnum{9}}[\unit{\decibel}]&\text{abs. Abw.}[\unit{\decibel}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{\num{-3.3(1.0)}&\text{\num{0.2(1.0)}&\text{\num{3.4(1.5)}&\text{\num{110(60)}&\text{\num{2.5}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_D_{\Rnum{3}}}
\end{table}
```

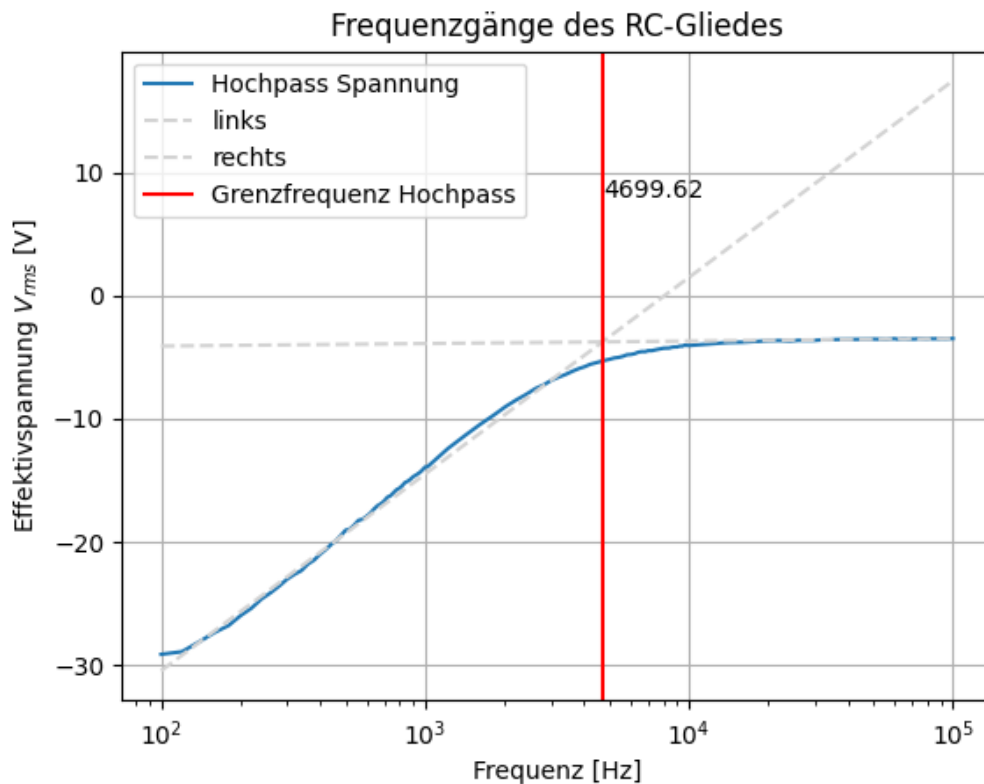
```
[ ]: plt.close('all')
#Grenzen für geraden Bereich
x0_tief=30
x1_tief=93
x0_hoch=30
x1_hoch=100
def single_log_linear(x, a, b):
    return a*np.log(x)+b
#Fit usw für Hochpass
#plot
plt.plot(freq_hoch, Vdb_hoch, label='Hochpass Spannung')
plt.xscale('log')
# plt.yscale('log')
plt.xlabel('Frequenz [Hz]')
plt.ylabel('Effektivspannung $V_{rms}$ [V]')
plt.legend()
```



```

#fit
popthoch_l,pcovhoch_l = curve_fit(single_log_linear, freq_hoch[:x0_hoch],
    ↪Vdb_hoch[:x0_hoch])
popthoch_r,pcovhoch_r = curve_fit(single_log_linear, freq_hoch[x1_hoch:],
    ↪Vdb_hoch[x1_hoch:])
#plotfit
plt.plot(freq_hoch, single_log_linear(freq_hoch, *popthoch_l), '--',
    ↪label='links',color='lightgrey')
plt.plot(freq_hoch, single_log_linear(freq_hoch, *popthoch_r), '--',
    ↪label='rechts',color='lightgrey')
#plt.plot(freq_tief, double_log_linear(freq_tief, *popttief), 'g--',
    ↪label='Tiefpass
#Plot Grenzfrequenz
a_l=popthoch_l[0]
b_l=popthoch_l[1]
a_r=popthoch_r[0]
b_r=popthoch_r[1]
f_cutoff_hoch = np.e**((b_r-b_l)/(a_l-a_r))
plt.axvline(x=f_cutoff_hoch, color='red', linestyle='-', label='Grenzfrequenz_
    ↪Hochpass')
plt.text(f_cutoff_hoch,8,f'{f_cutoff_hoch:.2f}')
#Fehlergeraden:
# plt.axvline(x=f_cutoff_hoch-f_cutoff_hoch, color='red', linestyle='--')
# plt.axvline(x=f_cutoff_hoch+f_cutoff_hoch.s, color='red', linestyle='--')
plt.legend()
plt.title('Frequenzgänge des RC-Gliedes')
plt.grid()
# plt.savefig('figures/A3/Frequenzgaenge_RC_hoch.png', dpi=300,
    ↪bbox_inches='tight')
plt.show()

```



-4.0814168011238205

```
[159]: R=np.array([1000,220,47])
Delta_R=R*0.05
C=np.array([47.0,47.0,47.0]) #nF
Delta_C=0.1*C
f_R=np.array([3.91,3.80,3.82]) #kHz
Delta_f_R=0.05
L_1=1/(4*np.pi**2*C*f_R**2)*1e6
Delta_L_1=L_1*np.sqrt(4*Delta_f_R**2/f_R**2 + Delta_C**2/C**2)
L_1,Delta_L_1,_,latexL_1=v_round(L_1,Delta_L_1,r'\milli\henry')
for i in range(3):
    print(rf'L_{1,R=\qty{{{R[i]}}}{\ohm}}}{\latexL_1[i]}')
print(np.std(L_1))
print(np.sqrt(np.std(L_1)**2+4**2))
L_1=np.mean(L_1)
Delta_L_1=4
L_1,Delta_L_1,_,latexbarL_1=round_to_sigs(L_1,Delta_L_1,r'\milli\henry')
print(latexbarL_1)
```

$L_{1,R=\qty{1000}{\ohm}}\qty{35(4)}{\milli\henry}$

$L_{1,R=\qty{220}{\ohm}}\qty{37(4)}{\milli\henry}$

L_{1,R}=\qty{47}{\ohm}\qty{37(4)}{\milli\henry}
 0.9428090415820634
 4.109609335312651
 \qty{36(4)}{\milli\henry}

[423]: `gff("1/(4*pi**2*f_R**2*C)","f_R,C")`

Funktion:

$$\frac{1}{4\pi^2 C f_R^2}$$

$\frac{1}{4\pi^2 C f_R^2}$

Absoluter Fehler:

$$\frac{1}{4\pi^2} \sqrt{\frac{1}{C^4 f_R^6} (4C^2 \Delta_{f_R}^2 + \Delta_C^2 f_R^2)}$$

$\frac{\sqrt{\frac{4 C^2 \Delta_{f_R}^2 + \Delta_C^2 f_R^2}{C^4 f_R^6}}}{4 \pi^2}$

Relativer Fehler:

$$\sqrt{\frac{4}{f_R^2} \Delta_{f_R}^2 + \frac{\Delta_C^2}{C^2}}$$

$\sqrt{\frac{4 \Delta_{f_R}^2}{f_R^2} + \frac{\Delta_C^2}{C^2}}$

[423]: `(1/(4*pi**2*C*f_R**2),
 sqrt((4*C**2*Delta_f_R**2 + Delta_C**2*f_R**2)/(C**4*f_R**6))/(4*pi**2),
 sqrt(4*Delta_f_R**2/f_R**2 + Delta_C**2/C**2),
 [(f_R, Delta_f_R), (C, Delta_C)])`

Diskussionsrechnung über Resfreq an C und L

[311]: `delta=1900
 Delta_delta=400
 f_C=3.82
 f_L=4.11
 f_R=4.00*1e3
 Delta_f_R=0.07*1e3
 f_ctheo=1/(2*np.pi)*np.sqrt((2*np.pi*f_R)**2-2*delta**2)
 f_Ltheo=1/(2*np.pi)*np.sqrt((2*np.pi*f_R)**2+2*delta**2)
 Delta_f_ctheo=np.sqrt(2)*np.sqrt((Delta_delta**2*delta**2 + 4*np.
 ↪pi**4*Delta_f_R**2*f_R**2)/(-delta**2 + 2*np.pi**2*f_R**2))/(2*np.pi)
 Delta_f_Ltheo=np.sqrt(2)*np.sqrt((Delta_delta**2*delta**2 + 4*np.
 ↪pi**4*Delta_f_R**2*f_R**2)/(delta**2 + 2*np.pi**2*f_R**2))/(2*np.pi)
 f_ctheo,Delta_f_ctheo,_,latexfctheo=round_to_sigs(f_ctheo,Delta_f_ctheo,r'\kilo\hertz')
 f_Ltheo,Delta_f_Ltheo,_,latexfLtheo=round_to_sigs(f_Ltheo,Delta_f_Ltheo,r'\kilo\hertz')
 print(latexfctheo)`

```

print(latexfLtheo)

compare_table(r'f^{5}',r'f^{theo}',r'\kilo\hertz',f_C,Delta_f_R*1e-3,f_ctheo*1e-3,Delta_f_ctheo)
compare_table(r'f^{5}',r'f^{theo}',r'\kilo\hertz',f_L,Delta_f_R*1e-3,f_Ltheo*1e-3,Delta_f_Ltheo)

\qty{3980(80)}{\kilo\hertz}
\qty{4020(80)}{\kilo\hertz}

\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
f^{5}[\unit{\kilo\hertz}]&f^{theo}[\unit{\kilo\hertz}]&\text{abs.Abw.}[\unit{\kilo\hertz}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\num{3.82(0.07)}&\num{3.98(0.08)}&\num{0.16(0.11)}&\num{4(3)}&\num{1.6}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_f^{5}}
\end{table}

\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
f^{5}[\unit{\kilo\hertz}]&f^{theo}[\unit{\kilo\hertz}]&\text{abs.Abw.}[\unit{\kilo\hertz}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\num{4.11(0.07)}&\num{4.02(0.08)}&\num{0.09(0.11)}&\num{2(3)}&\num{0.9}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_f^{5}}
\end{table}

```

[294]: `gff("1/(2*pi)*sqrt((2*pi*f_R)**2+2*delta**2)","f_R,delta")`

Funktion:

$$\frac{1}{2\pi} \sqrt{2\delta^2 + 4\pi^2 f_R^2}$$

$$\frac{\sqrt{2 \delta^2 + 4 \pi^2 f_R^2}}{2 \pi}$$

Absoluter Fehler:

$$\frac{\sqrt{2}}{2\pi} \sqrt{\frac{\Delta_\delta^2 \delta^2 + 4\pi^4 \Delta_{f_R}^2 f_R^2}{\delta^2 + 2\pi^2 f_R^2}}$$

$$\frac{\sqrt{2}}{2\pi} \sqrt{\frac{\Delta_{\delta}^2 \delta^2 + 4 \pi^4 \Delta_{f_R}^2 f_R^2}{\delta^2 + 4 \pi^2 f_R^2}}$$

$$R^2 f_R^2 \{ \delta^2 + 2 \pi^2 f_R^2 \}^2 \pi$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_\delta^2 \delta^2 + 4\pi^4 \Delta_{f_R}^2 f_R^2}{(\delta^2 + 2\pi^2 f_R^2)^2}}$$

$$\sqrt{\frac{\Delta_{\delta}^2 \delta^2 + 4 \pi^4 \Delta_{f_R}^2 f_R^2}{f_R^2 \left(\delta^2 + 2 \pi^2 f_R^2 \right)^2}}$$

```
[294]: (sqrt(2*delta**2 + 4*pi**2*f_R**2)/(2*pi),
        sqrt(2)*sqrt((Delta_delta**2*delta**2 + 4*pi**4*Delta_f_R**2*f_R**2)/(delta**2
+ 2*pi**2*f_R**2))/(2*pi),
        sqrt((Delta_delta**2*delta**2 + 4*pi**4*Delta_f_R**2*f_R**2)/(delta**2 +
2*pi**2*f_R**2)**2),
        [(f_R, Delta_f_R), (delta, Delta_delta)])
```

Komische Frequenzgänge miteinander verglichen

```
[395]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.56,6)))
plt.title('alle Frequenzgänge')
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'$|U_{eff}|$ [Vrms]')

f1,dBV1,_ = np.loadtxt(Messwerte_path/"nr1.txt",skiprows=1, unpack=True)
f2,dBV2,_ = np.loadtxt(Messwerte_path/"nr2.txt",skiprows=1, unpack=True)
f3,dBV3,_ = np.loadtxt(Messwerte_path/"nr3.txt",skiprows=1, unpack=True)
f4,dBV4,_,_ = np.loadtxt(Messwerte_path/"nr4.txt",skiprows=1, unpack=True)
f5,dBV5,_,_ = np.loadtxt(Messwerte_path/"nr5.txt",skiprows=1, unpack=True)
f6,dBV6,_ = np.loadtxt(Messwerte_path/"nr6.txt",skiprows=1, unpack=True)

# T=T/(1.25*1e6)
# U_A=(U_A-126)/32
# U_E=(U_E-132)/32

plt.plot(f1,dBV1,color='green',label=rf'Aufg.5, Kondensator C')
plt.plot(f2,dBV2,color='blue',label=rf'Aufg.5, Spule L')
plt.plot(f3,dBV3,color='orange',label=rf'Aufg.5, R=220$\Omega$ ')
plt.plot(f6,dBV6,color='red',label=rf'Aufg.4, R=220$\Omega$')
plt.plot(f4,dBV4,color='black',label=rf'Aufg.4, R=1000$\Omega$')
plt.plot(f5,dBV5,color='blue',label=rf'Aufg.4, R=47$\Omega$')

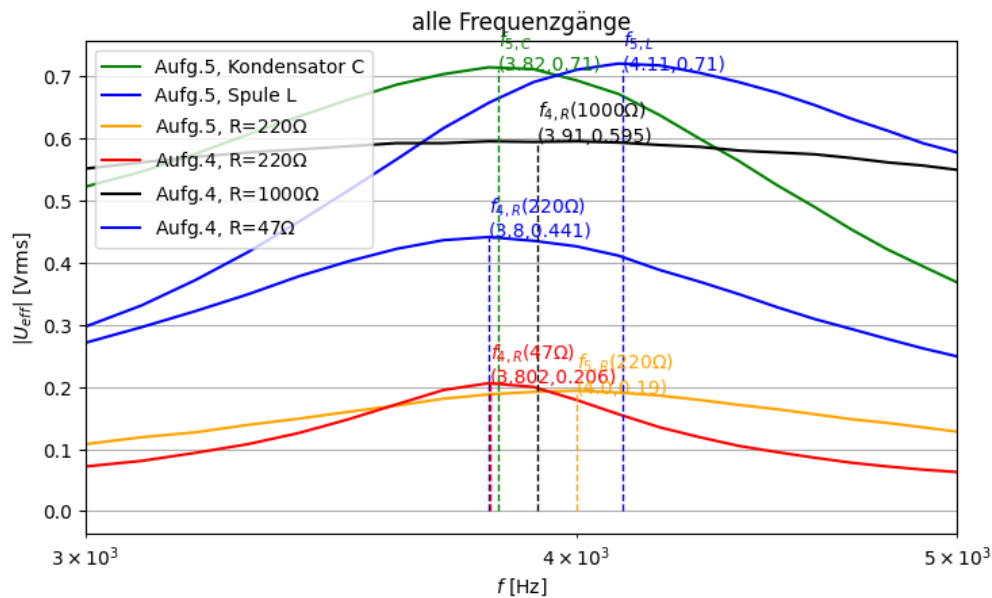
# plt.vlines(x=[4000,3800], ymin=0, ymax=[0.19,0.206],
#           color='black',linewidth=1,linestyle='--')
# plt.vlines(x=[3820,4110,3820], ymax=0.9, ymin=[0.72,0.72,0.441],
#           color='black',linewidth=1,linestyle='--')
```

```
# plt.vlines(x=[3820,4000,4110,3800,3820], ymin=0, ymax=[0.72,0.19,0.72,0.206,0.
↪441], color='black',linewidth=1,linestyle='--')

freq= { # in dieser Liste
    ↪werden die Namen der einzuzeichnenden Linien gespeichert
    r'$f_{4,R} (220\Omega)$': ('blue',[3800,0.441]),
    r'$f_{4,R} (47\Omega)$': ('red',[3802,0.206]),
    r'$f_{4,R} (1000\Omega)$': ('black',[3910,0.595]),
    r'$f_{5,R} (220\Omega)$': ('orange',[4000,0.19]),
    r'$f_{5,L} $': ('blue',[4110,0.71]),
    r'$f_{5,C} $': ('green',[3820,0.71]),}

for name, (color,f) in freq.items():
    plt.text(f[0],f[1],f"{name}\n({f[0]/1000},{f[1]})",color=color)
    plt.vlines(x=f[0], ymax=f[1], ymin=0,
    ↪color=color,linewidth=1,linestyle='--')

plt.xscale('log')
plt.xlim(3e3,5e3)
plt.legend(loc='upper left')
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path/"Diskussion_überhöhung.png",format="png")
plt.show()
```



V Verlustwiderstand über Δf

```
[21]: Delta_f=np.array([4.87,1.38,0.63])
Delta_Delta_f=0.05*np.sqrt(2)
R_ges=2*pi*Delta_f*L_1
Delta_R_ges= R_ges*np.sqrt(Delta_Delta_f**2/Delta_f**2 + Delta_L_1**2/L_1**2)
R_ges,Delta_R_ges,latexRges,_=v_round(R_ges,Delta_R_ges,r'\ohm')
for i in range(3):
    print(rf'R_{\text{{ges}},R=\qty{{{R[i]}}}{\ohm}}}{\latexRges[i]}')
R_Vges=R_ges-R
Delta_R_Vges=np.sqrt(Delta_R**2+Delta_R_ges**2)
R_Vges,Delta_R_Vges,latexR_Vges,_=v_round(R_Vges,Delta_R_Vges,r'\ohm')
```

R_{\text{{ges}},R=\qty{1000}{\ohm}}\num{1100(130)}
R_{\text{{ges}},R=\qty{220}{\ohm}}\num{310(40)}
R_{\text{{ges}},R=\qty{47}{\ohm}}\num{143(23)}

```
[ ]: gff("R_ges-R", "R_ges,R")
```

Funktion:

$$-R + R_{ges}$$

$$- R + R_{\text{{ges}}}$$

Absoluter Fehler:

$$\sqrt{\Delta_R^2 + \Delta_{R_{ges}}^2}$$

$$\sqrt{\Delta_{\text{{R}}}^2 + \Delta_{\text{{R ges}}}^2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2 + \Delta_{R_{ges}}^2}{(R - R_{ges})^2}}$$

$$\sqrt{\frac{\Delta_{\text{{R}}}^2 + \Delta_{\text{{R ges}}}^2}{\left(R - R_{\text{{ges}}}\right)^2}}$$

```
[ ]: (-R + R_ges,
      sqrt(Delta_R**2 + Delta_R_ges**2),
      sqrt((Delta_R**2 + Delta_R_ges**2)/(R - R_ges)**2),
      [(R_ges, Delta_R_ges), (R, Delta_R)])
```

```
[ ]: gff("2*pi*Delta_f*L", "Delta_f,L")
```

Funktion:

$$2\pi\Delta_fL$$

$$2 \pi \Delta_{\text{{f}}} L$$

Absoluter Fehler:

$$2\pi\sqrt{\Delta_{\Delta_f}^2L^2 + \Delta_L^2\Delta_f^2}$$

$2 \pi \sqrt{\Delta_f^2 L^2 + \Delta_L^2 \Delta_f^2}$
 Relativer Fehler:

$$\sqrt{\frac{\Delta_f^2}{\Delta_f^2} + \frac{\Delta_L^2}{L^2}}$$

$\sqrt{\frac{\Delta_f^2}{\Delta_f^2} + \frac{\Delta_L^2}{L^2}}$
 $\sqrt{\frac{\Delta_f^2}{\Delta_f^2} + \frac{\Delta_L^2}{L^2}}$

```
[ ]: (2*pi*Delta_f*L,
      2*pi*sqrt(Delta_Delta_f**2*L**2 + Delta_L**2*Delta_f**2),
      sqrt(Delta_Delta_f**2/Delta_f**2 + Delta_L**2/L**2),
      [(Delta_f, Delta_Delta_f), (L, Delta_L)])
```

Verlustwiderstand über U

```
[424]: U_E=0.71
Delta_U_E=0.01
U_A=np.array([0.595,0.441,0.206])
Delta_U_A=0.01
R=np.array([1000,220,47])
Delta_R=0.05*R
R,Delta_R,latexR,_=v_round(R,Delta_R,r'\ohm')
R_V=(U_E/U_A-1)*R
Delta_R_V= np.sqrt((Delta_U_A**2*R**2*U_E**2 + U_A**2*(Delta_R**2*(-U_A +
↪U_E)**2 + Delta_U_E**2*R**2))/U_A**4)
R_V,Delta_R_V,latexR_V,_=v_round(R_V,Delta_R_V,r'\ohm')
for i in range(3):
    ↵
    ↪print(rf'{latexR[i]}&{latexR_Vges[i]}&{latexR_V[i]}&{sigma_abweichung(R_Vges[i],Delta_R_Vge

compare_table_array(r'',r'',r'',R_Vges,Delta_R_Vges,R_V,Delta_R_V)
```

```
\num{1000(50)}&\num{100(140)}&\num{190(30)}&\num{0.7}\\
\num{220(11)}&\num{90(50)}&\num{134(12)}&\num{0.9}\\
\num{47.0(2.4)}&\num{96(24)}&\num{115(11)}&\num{0.8}\\
\begin{table}[htb]
\centering
\caption{Vergleich von $$ und $$}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe & [\unit{}] & [\unit{}] & \text{abs.Abw.}[\unit{}] &
\text{proz.Abw.}[\unit{\percent}] & S[\sigma] \\
\midrule
& \num{100(140)} & \num{190(30)} & \num{90(150)} & \num{90(200)} &
\num{0.7} \\
& \num{90(50)} & \num{134(12)} & \num{40(60)} & \num{50(70)} & \num{0.9} \\
& \num{96(24)} & \num{115(11)} & \num{20(30)} & \num{20(30)} & \num{0.8} \\
\end{tabular}
\end{table}
```



```
\bottomrule
\end{tabularx}
\label{table:Vergleich_}
\end{table}
```

```
[ ]: gff("(U_E/U_A-1)*R", "U_E,U_A,R")
```

Funktion:

$$R \left(-1 + \frac{U_E}{U_A} \right)$$

$R \left(-1 + \frac{U_E}{U_A} \right)$

Absoluter Fehler:

$$\sqrt{\frac{1}{U_A^4} \left(\Delta_{UA}^2 R^2 U_E^2 + U_A^2 \left(\Delta_R^2 (-U_A + U_E)^2 + \Delta_{UE}^2 R^2 \right) \right)}$$

$\sqrt{\frac{\Delta_{UA}^2 R^2 U_E^2 + U_A^2 \left(\Delta_R^2 (-U_A + U_E)^2 + \Delta_{UE}^2 R^2 \right)}{U_A^4}}$

Relativer Fehler:

$$\sqrt{\frac{1}{R^2 U_A^2 (U_A - U_E)^2} \left(\Delta_{UA}^2 R^2 U_E^2 + U_A^2 \left(\Delta_R^2 (-U_A + U_E)^2 + \Delta_{UE}^2 R^2 \right) \right)}$$

$\sqrt{\frac{\Delta_{UA}^2 R^2 U_E^2 + U_A^2 \left(\Delta_R^2 (-U_A + U_E)^2 + \Delta_{UE}^2 R^2 \right)}{R^2 U_A^2 (U_A - U_E)^2}}$

```
[ ]: (R*(-1 + U_E/U_A),
      sqrt((Delta_U_A**2*R**2*U_E**2 + U_A**2*(Delta_R**2*(-U_A + U_E)**2 +
Delta_U_E**2*R**2))/U_A**4),
      sqrt((Delta_U_A**2*R**2*U_E**2 + U_A**2*(Delta_R**2*(-U_A + U_E)**2 +
Delta_U_E**2*R**2))/(R**2*U_A**2*(U_A - U_E)**2)),
      [(U_E, Delta_U_E), (U_A, Delta_U_A), (R, Delta_R)])
```

VI Resonanzüberhöhung

```
[320]: f_L=4.11 #kHz
Delta_f_L=Delta_f_R=np.sqrt(2)*0.05
f_R=4.0
R=220
Delta_R=0.05*R
L_1=np.sqrt(2)*R/(4*np.pi*np.sqrt(f_L**2 - f_R**2))
Delta_L_1=L_1*np.sqrt(Delta_R**2/R**2 + Delta_f_L**2*f_L**2/(f_L**2 -
↪f_R**2)**2 + Delta_f_R**2*f_R**2/(f_L**2 - f_R**2)**2)
L_1,Delta_L_1,_,latexL1=round_to_sigs(L_1,Delta_L_1,r'\henry')
print(latexL1)

f_C=3.82 #kHz
```

```

Delta_f_C=Delta_f_R=np.sqrt(2)*0.05
f_R=4.0
R=220
Delta_R=0.05*R
L_2=np.sqrt(2)*R/(4*np.pi*np.sqrt(-(f_C**2 - f_R**2)))
Delta_L_2=L_2*np.sqrt(Delta_R**2/R**2 + Delta_f_C**2*f_C**2/(-(f_C**2 -
↪f_R**2))**2 + Delta_f_R**2*f_R**2/(-(f_C**2 - f_R**2))**2)
L_2,Delta_L_2,_,latexL2=round_to_sigs(L_2,Delta_L_2,r'\henry')
print(latexL2)
print(Delta_f_R)

```

```

\qty{26(12)}{\henry}
\qty{21(6)}{\henry}
0.07071067811865477

```

[45]: `gff("-R/sqrt(8*pi**2*(f_C**2-f_R**2))", "R,f_C,f_R")`

Funktion:

$$-\frac{\sqrt{2}R}{4\pi\sqrt{f_C^2 - f_R^2}}$$

- $\frac{\sqrt{2} R}{4 \pi \sqrt{f_{\text{C}}^2 - f_{\text{R}}^2}}$

Absoluter Fehler:

$$\frac{\sqrt{2}}{4\pi} \sqrt{\frac{1}{f_C^2 - f_R^2} \left(\Delta_R^2 + \frac{R^2}{(f_C^2 - f_R^2)^2} (\Delta_{f_C}^2 f_C^2 + \Delta_{f_R}^2 f_R^2) \right)}$$

$\frac{\sqrt{2}}{4\pi} \sqrt{\frac{\Delta_{\text{R}}^2}{R^2} + \frac{\Delta_{\text{f C}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} + \frac{\Delta_{\text{f R}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} \left(\frac{\Delta_{\text{f C}}^2 f_{\text{C}}^2 + \Delta_{\text{f R}}^2 f_{\text{R}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} \right)}$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_{f_C}^2 f_C^2}{(f_C^2 - f_R^2)^2} + \frac{\Delta_{f_R}^2 f_R^2}{(f_C^2 - f_R^2)^2}}$$

$\sqrt{\frac{\Delta_{\text{R}}^2}{R^2} + \frac{\Delta_{\text{f C}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} + \frac{\Delta_{\text{f R}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} \left(\frac{\Delta_{\text{f C}}^2 f_{\text{C}}^2 + \Delta_{\text{f R}}^2 f_{\text{R}}^2}{(f_{\text{C}}^2 - f_{\text{R}}^2)^2} \right)}$

[45]: `(-sqrt(2)*R/(4*pi*sqrt(f_C**2 - f_R**2)),
sqrt(2)*sqrt((Delta_R**2 + R**2*(Delta_f_C**2*f_C**2 +
Delta_f_R**2*f_R**2)/(f_C**2 - f_R**2)**2)/(f_C**2 - f_R**2))/(4*pi),
sqrt(Delta_R**2/R**2 + Delta_f_C**2*f_C**2/(f_C**2 - f_R**2)**2 +
Delta_f_R**2*f_R**2/(f_C**2 - f_R**2)**2),
[(R, Delta_R), (f_C, Delta_f_C), (f_R, Delta_f_R)])`

Vergleich Induktivitäten

```
[60]: L_1RC=np.array([26,21])
Delta_L_1RC=np.array([12,6])
L_1=np.array([36,36])
Delta_L_1=np.array([4,4])
compare_table_array(r'',r'',r'\milli\henry',L_1RC,Delta_L_1RC,L_1,Delta_L_1)

\begin{table}[htb]
\centering
\caption{Vergleich von $$ und $$}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe & [\unit{\milli\henry}] & [\unit{\milli\henry}] & 
\text{abs.Abw.}[\unit{\milli\henry}] & \text{proz.Abw.}[\unit{\percent}] & 
S[\sigma] \\ \midrule
& \num{26(12)} & \num{36(4)} & \num{10(13)} & \num{40(60)} & \num{0.8} \\
\\
& \num{21(6)} & \num{36(4)} & \num{15(8)} & \num{70(40)} & \num{2.1} \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_}
\end{table}
```

VII Damped RLC

```
[75]: A_N=np.array([3.69,2.53,1.75,1.25,1.06])
Delta_A_N=0.1
Lambda=[]
Delta_Lambda=[]
for i in range(len(A_N)-1):
    Lambda.append(np.log(A_N[i]/A_N[i+1]))
    Delta_Lambda.append(Delta_A_N*np.sqrt(1/A_N[i]**2+1/A_N[i+1]**2))
Lambda=np.array(Lambda)
Delta_Lambda=np.array(Delta_Lambda)
print(Lambda)
Lambda,Delta_Lambda,latexLambda,_=round_to_sigs(np.mean(Lambda),np.
    ↪mean(Delta_Lambda),r'')
print(latexLambda)
```

```
[0.37740716 0.36860351 0.33647224 0.16487464]
\num{0.31(0.09)}
```

```
[92]: plt.close('all')
plt.figure(figsize=(figsize_cm(12,6)))
plt.title('Gedämpfter RLC-Schwingkreis')
plt.xlabel(r't[ms]')
plt.ylabel(r'UA[V]')

t=np.array([1,2,3,4,5])*0.76/3
```

```

Delta_t=np.sqrt(2)*0.1/3
plt.errorbar(t, A_N, yerr=Delta_A_N,xerr=Delta_t,fmt=".",capsize=3,elinewidth=0.
↪5,label='gemessene Maxima')

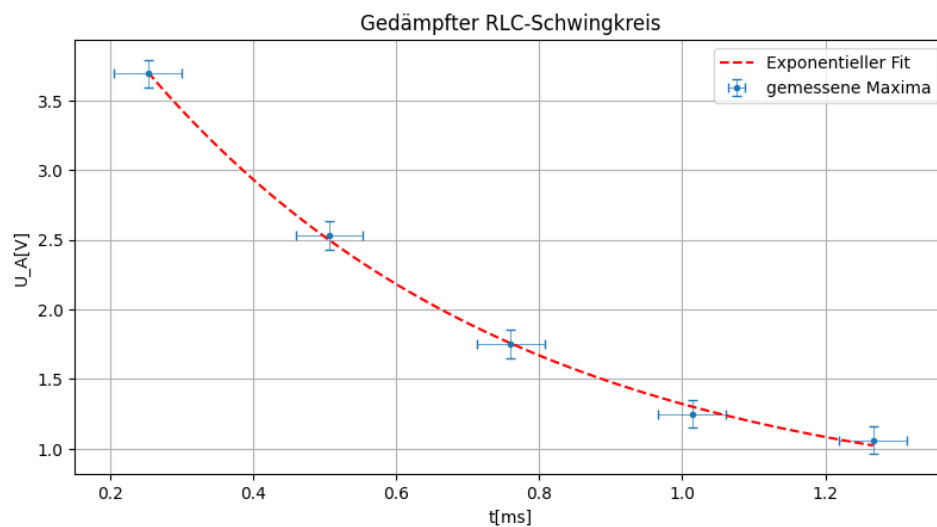
def exp_fit(t, a, b, c):
    return a * np.exp(-b * t) + c

popt,pcov=curve_fit(exp_fit, t, A_N, sigma=Delta_A_N, absolute_sigma=True)
times = np.linspace(min(t), max(t), 100)
plt.plot(times, exp_fit(times, *popt), 'r--', label='Exponentieller Fit')

plt.legend()
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path/"AVII_dämpfung.png",format="png")
plt.show()

delta,Delta_delta,_,latexdelta=round_to_sigs(popt[1]*1e3, np.
↪sqrt(pcov[1][1])*1e3,r'\per\s')
print(latexdelta)

```



$\sqrt[1900]{400} \text{ per s}$

VII gedämpfter RLC R_{ges}

```

[96]: L=36*1e-3
Delta_L=4*1e-3
delta=1900
Delta_delta=400

```

```

R=47
Delta_R=0.05*R
R_delta_ges=2*L*delta-R
Delta_R_delta_ges=np.sqrt(4*Delta_L**2*delta**2 + Delta_R**2 +
↪4*Delta_delta**2*L**2)
R_delta_ges,Delta_R_delta_ges,_,latexRdeltages=round_to_sigs(R_delta_ges,Delta_R_delta_ges,r'\
print(latexRdeltages)
print(sigma_abweichung(90,40,96,24))

```

```

\qty{90(40)}{\ohm}
(0.13, '\num{0.13}')

```

```

[97]: L_1RC=np.array([96,115])
Delta_L_1RC=np.array([24,11])
L_1=np.array([90,90])
Delta_L_1=np.array([40,40])
compare_table_array(r'',r'',r'\ohm',L_1RC,Delta_L_1RC,L_1,Delta_L_1)

```

```

\begin{table}[htb]
\centering
\caption{Vergleich von $$ und $$}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe & [\unit{\ohm}] & [\unit{\ohm}] & & \\
\text{abs. Abw.} [\unit{\ohm}] & \text{proz. Abw.} [\unit{\percent}] & S[\sigma] & & \\
\\
& \num{96(24)} & \num{90(40)} & \num{10(50)} & \num{10(50)} & \num{0.13} \\
\\
& \num{115(11)} & \num{90(40)} & \num{30(50)} & \num{20(40)} & \num{0.7} \\
\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_}
\end{table}

```

```

[118]: gff("2*L*delta-R", "L,delta,R")

```

Funktion:

$$2L\delta - R$$

$$2L\delta - R$$

Absoluter Fehler:

$$\sqrt{4\Delta_L^2\delta^2 + \Delta_R^2 + 4\Delta_\delta^2L^2}$$

$$\sqrt{4\Delta_L^2\delta^2 + \Delta_R^2 + 4\Delta_\delta^2L^2}$$

Relativer Fehler:

$$\sqrt{\frac{4\Delta_L^2\delta^2 + \Delta_R^2 + 4\Delta_\delta^2 L^2}{(2L\delta - R)^2}}$$

$\sqrt{\frac{4 \Delta_L^2 \delta^2 + \Delta_R^2 + 4 \Delta_\delta^2 L^2}{L^2}} \sqrt{\left(2 L \delta - R\right)^2}$

```
[118]: (2*L*delta - R,
        sqrt(4*Delta_L**2*delta**2 + Delta_R**2 + 4*Delta_delta**2*L**2),
        sqrt((4*Delta_L**2*delta**2 + Delta_R**2 + 4*Delta_delta**2*L**2)/(2*L*delta -
        R)**2),
        [(L, Delta_L), (delta, Delta_delta), (R, Delta_R)])
```

VII damped RLC Periodendauer

```
[337]: print(round_to_sigs(0.76/3,sqrt(2)*0.1/3,r'\ms')[3])
f=1/0.25
Delta_f=0.05/0.25**2
print(f,Delta_f)
# f,Delta_f,_,latexf=round_to_sigs(f,Delta_f,r'\kilo\hertz')
# print(latexf)
print(sigma_abweichung(4,0.8,3.82,0.07))
```

$\sqrt{0.25(0.05)} \text{ ms}$

4.0 0.8

(0.23, '\num{0.23}')

Resonanzfrequenz aus T richtig

```
[336]: delta=1900
Delta_delta=400
f_d=4*1e3
Delta_f_d=0.8*1e3
f_0=np.sqrt(-delta**2 + 4*np.pi**2*f_d**2)/(2*np.pi)
Delta_f_0= np.sqrt((Delta_delta**2*delta**2 + 16*np.pi**4*Delta_f_d**2*f_d**2)/
    (-delta**2 + 4*np.pi**2*f_d**2))/(2*np.pi)
# f_0,Delta_f_0,_,latexf0=round_to_sigs(f_0,Delta_f_0,r'\hertz')
# print(latexf0)
print(f_0,Delta_f_0)
```

3988.5533256762146 802.3104230378312

```
[332]: gff("1/(2*pi)*sqrt((2*pi*f_d)**2-delta**2)","f_d,delta")
```

Funktion:

$$\frac{1}{2\pi} \sqrt{-\delta^2 + 4\pi^2 f_d^2}$$

$\frac{\sqrt{-\delta^2 + 4 \pi^2 f_d^2}}{2 \pi}$

Absoluter Fehler:

$$\frac{1}{2\pi} \sqrt{\frac{\Delta_\delta^2 \delta^2 + 16\pi^4 \Delta_{fd}^2 f_d^2}{-\delta^2 + 4\pi^2 f_d^2}}$$

$\frac{\sqrt{\frac{\Delta_{\delta}^2 \delta^2 + 16 \pi^4 \Delta_{fd}^2 f_d^2}{-\delta^2 + 4 \pi^2 f_d^2}}}{f_d^2} \{- \delta^2 + 4 \pi^2 f_d^2\}^{\frac{1}{2}} \pi$

Relativer Fehler:

$$\sqrt{\frac{\Delta_\delta^2 \delta^2 + 16\pi^4 \Delta_{fd}^2 f_d^2}{(\delta^2 - 4\pi^2 f_d^2)^2}}$$

$\frac{\sqrt{\frac{\Delta_{\delta}^2 \delta^2 + 16 \pi^4 \Delta_{fd}^2 f_d^2}{f_d^2}}}{\left(\delta^2 - 4 \pi^2 f_d^2 \right)^{\frac{1}{2}}}$

```
[332]: (sqrt(-delta**2 + 4*pi**2*f_d**2)/(2*pi),
        sqrt((Delta_delta**2*delta**2 + 16*pi**4*Delta_f_d**2*f_d**2)/(-delta**2 +
        4*pi**2*f_d**2))/(2*pi),
        sqrt((Delta_delta**2*delta**2 + 16*pi**4*Delta_f_d**2*f_d**2)/(delta**2 -
        4*pi**2*f_d**2)**2),
        [(f_d, Delta_f_d), (delta, Delta_delta)])
```

```
[121]: compare_table(r'f_R\text{exp}',r'f_R\text{theo}',r'\kilo\hertz',3.82,0.07,4.
        ↪0,0.8)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
f_R\text{exp}[\unit{\kilo\hertz}]&f_R\text{theo}[\unit{\kilo\hertz}]&\text{abs. Abw.}[\unit{\kilo\hertz}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\
\midrule
\num{3.82(0.07)}&\num{4.0(0.8)}&\num{0.2(0.9)}&\num{5(22)}&\num{0.23}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_f_R\text{exp}}
\end{table}
```

VIII.Parallelschwingkreis

```
[ ]: L=36*1e-3
Delta_L=4*1e-3
C=47*1e-9 #nF
Delta_C=0.1*C
L=1/(2*np.pi*sqrt(C*L))
Delta_L=L*np.sqrt(Delta_L**2/L**2 + Delta_C**2/C**2)/2
L,Delta_L,_,latexL=round_to_sigs(L,Delta_L,r'\hertz')
print(latexL)
```

\qty{3870(200)}{\hertz}
(0.05, '\num{0.05}')

[338]: `compare_table(r'f_R\text{exp}',r'f_R\text{theo}',r'\kilo\hertz',3.88,0.1,3.
↪87,0.2)`

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
f_R\text{exp}[\unit{\kilo\hertz}]&f_R\text{theo}[\unit{\kilo\hertz}]&\text{abs.Abw.}[\unit{\kilo\hertz}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\text{\\}
\midrule
\num{3.88(0.10)}&\num{3.87(0.20)}&\num{0.01(0.23)}&\num{0(6)}&\num{0.05}\text{\\}
\bottomrule
\end{tabularx}
\label{table:Vergleich_f_R\text{exp}}
\end{table}
```

[104]: `gff("1/(2*pi*sqrt(L*C))", "L,C")`

Funktion:

$$\frac{1}{2\pi\sqrt{CL}}$$

`\frac{1}{2 \pi \sqrt{C L}}`

Absoluter Fehler:

$$\frac{\sqrt{\frac{\Delta_L^2}{L^2} + \frac{\Delta_C^2}{C^2}}}{4\pi}$$

`\frac{\sqrt{\frac{\Delta_{L}^{\wedge}2}{L^{\wedge}2} + \frac{\Delta_{C}^{\wedge}2}{C^{\wedge}2}}}{4 \pi}`

Relativer Fehler:

$$\frac{1}{2}\sqrt{\frac{\Delta_L^2}{L^2} + \frac{\Delta_C^2}{C^2}}$$

`\frac{\sqrt{\frac{\Delta_{L}^{\wedge}2}{L^{\wedge}2} + \frac{\Delta_{C}^{\wedge}2}{C^{\wedge}2}}}{2}`

[104]: `(1/(2*pi*sqrt(C*L)),
sqrt((Delta_L**2/L**2 + Delta_C**2/C**2)/(C*L))/(4*pi),
sqrt(Delta_L**2/L**2 + Delta_C**2/C**2)/2,
[(L, Delta_L), (C, Delta_C)])`

Decibel Skala


```
[148]: numbers=np.array([40,30,20,10,6,3,0,-3,-6,-10,-20,-30,-40])
dB=np.round(10**(numbers/20),2)
dB=np.sort(dB)
numbers=np.sort(numbers)
text =" & ".join(numbers.astype(str))
text2 = " & ".join(dB.astype(str))
print(text)
print(text2)
print(len(dB))
print(len(numbers))
print(10**(-10/20))
```

```
-40 & -30 & -20 & -10 & -6 & -3 & 0 & 3 & 6 & 10 & 20 & 30 & 40
0.01 & 0.03 & 0.1 & 0.32 & 0.5 & 0.71 & 1.0 & 1.41 & 2.0 & 3.16 & 10.0 & 31.62 &
100.0
13
13
0.31622776601683794
```